접근 제어 및 콘텐츠 차단 시스템

개요

IPFS의 접근 제어 및 콘텐츠 차단 시스템은 분산 네트워크에서 보안 정책을 구현하고 불법적이거나 유해한 콘텐츠에 대한 접근을 제한하는 메커니즘을 제공합니다. 이 시스템은 API 접근 제어, 게이트웨이 보안, 콘텐츠 필터링, 그리고 피어 관리를 포함한 다층적 보안 접근 방식을 사용합니다.

API 접근 제어 시스템

RPC API 인증 및 권한 관리

IPFS는 RPC API에 대한 세밀한 접근 제어를 제공합니다:

1. 인증 스키마 구성

```
// config/api.go의 RPCAuthScope 구조
type RPCAuthScope struct {
// 인증 시크릿 (type:value 형식)
AuthSecret  string
// 허용된 API 경로 목록
AllowedPaths []string
}
```

```
type API struct {
```

```
// HTTP 헤더 설정

HTTPHeaders  map[string][]string

// 사용자별 권한 설정

Authorizations  map[string]*RPCAuthScope

}
```

2. 지원되는 인증 방식

Bearer Token 인증

```
{

"API":  {

"Authorizations":  {

"user1":  {

"AuthSecret":  "bearer:your-secret-token-here",

"AllowedPaths":  ["/api/v0/add",  "/api/v0/cat"]

}

}

}

}
```

Basic 인증

```
{

"API":  {
```

```
"Authorizations": {
"admin": {
"AuthSecret": "basic:username:password",
"AllowedPaths": ["/api/v0"]
}
}
}
}
```

3. 인증 구현 세부사항

```
// core/corehttp/commands.go의 인증 처리
func withAuthSecrets(authorizations map[string]rpcAuthScopeWithUser, next http
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
authorizationHeader := r.Header.Get("Authorization")
auth, ok := authorizations[authorizationHeader]

if ok {
// 버전 확인은 항상 허용
if r.URL.Path == "/api/v0/version" {
next.ServeHTTP(w, r)
return
}
// 허용된 경로 확인
```

```
for _, prefix := range auth.AllowedPaths {
if strings.HasPrefix(r.URL.Path, prefix) {
next.ServeHTTP(w, r)
return
}
}
}

http.Error(w, "Kubo RPC Access Denied: Please provide a valid authorization toke
})
}
```

4. 권한 레벨 설계

권한 레벨	허용 경로	설명
읽기 전용	`/api/v0/cat`, `/api/v0/ls`, `/api/v0/dag/get`	콘텐츠 조회만 가능
기본 사용자	`/api/v0/add`, `/api/v0/pin`	콘텐츠 추가 및 핀 관리
고급 사용자	`/api/v0/swarm`, `/api/v0/dht`	네트워크 관리 기능
관리자	`/api/v0`	모든 API 접근 가능

CORS (Cross-Origin Resource Sharing) 설정

1. 기본 CORS 정책

```go
// core/corehttp/commands.go의 CORS 설정
var defaultLocalhostOrigins = []string{
"http://127.0.0.1:<port>",
"https://127.0.0.1:<port>",
"http://[::1]:<port>",
"https://[::1]:<port>",
"http://localhost:<port>",
"https://localhost:<port>",
}

var companionBrowserExtensionOrigins = []string{
"chrome-extension://nibjojkomfdiaoajekhjakgkdhaomnch", // ipfs-companion
"chrome-extension://hjoieblefckbooibpepigmacodalfndh", // ipfs-companion-beta
}
```

2. CORS 헤더 관리

```go
func addHeadersFromConfig(c *cmdsHttp.ServerConfig, nc *config.Config) {
// Access-Control-Allow-Origin 설정
if acao := nc.API.HTTPHeaders[cmdsHttp.ACAOrigin]; acao != nil {
c.SetAllowedOrigins(acao...)
}
// Access-Control-Allow-Methods 설정
```

```
if acam := nc.API.HTTPHeaders[cmdsHttp.ACAMethods]; acam != nil {
c.SetAllowedMethods(acam...)
}
// Access-Control-Allow-Credentials 설정
for _, v := range nc.API.HTTPHeaders[cmdsHttp.ACACredentials] {
c.SetAllowCredentials(strings.ToLower(v) == "true")
}
}
```

3. 동적 포트 처리

```
func patchCORSVars(c *cmdsHttp.ServerConfig, addr net.Addr) {
port := ""
if tcpaddr, ok := addr.(*net.TCPAddr); ok {
port = strconv.Itoa(tcpaddr.Port)
}

// <port> 플레이스홀더를 실제 포트로 교체
oldOrigins := c.AllowedOrigins()
newOrigins := make([]string, len(oldOrigins))
for i, o := range oldOrigins {
if port != "" {
o = strings.Replace(o, "<port>", port, -1)
```

```
}

newOrigins[i] = o

}

c.SetAllowedOrigins(newOrigins...)

}
```

게이트웨이 보안 및 접근 제어

게이트웨이 설정 구조

1. 전역 게이트웨이 설정

```
// config/gateway.go의 Gateway 구조

type Gateway struct {

// HTTP 헤더 설정

HTTPHeaders  map[string][]string

// 루트 리다이렉트 경로

RootRedirect  string

// 블록 페치 비활성화 (Trustless Gateway)

NoFetch  bool

// DNS 링크 비활성화

NoDNSLink  bool

// 역직렬화 응답 설정

DeserializedResponses Flag
```

```
// HTML 오류 페이지 비활성화

DisableHTMLErrors Flag

// 공개 게이트웨이 설정

PublicGateways  map[string]*GatewaySpec

// 검색 타임아웃

RetrievalTimeout *OptionalDuration

// 최대 동시 요청 수

MaxConcurrentRequests *OptionalInteger

}
```

2. 도메인별 게이트웨이 설정

```
type GatewaySpec struct {

// 허용된 경로 목록

Paths []string

// 서브도메인 사용 여부

UseSubdomains  bool

// DNS 링크 비활성화

NoDNSLink  bool

// 인라인 DNS 링크

InlineDNSLink Flag

// 역직렬화 응답 허용

DeserializedResponses Flag
```

```
}
```

게이트웨이 보안 정책

1. Trustless Gateway 모드

```
// core/corehttp/gateway.go의 Trustless 설정
func newGatewayBackend(n *core.IpfsNode) (gateway.IPFSBackend, error) {
cfg, err := n.Repo.Config()
if err != nil {
return nil, err
}

if cfg.Gateway.NoFetch {
// 오프라인 모드: 로컬 블록만 제공
bserv = blockservice.New(bserv.Blockstore(), offline.Exchange(bserv.Blockstore()
// 오프라인 경로 해석기 사용
pathResolver = n.OfflineUnixFSPathResolver
}
}
```

2. 요청 제한 및 속도 제한

```
// 최대 동시 요청 수 제한
```

```
gwConfig := gateway.Config{
MaxConcurrentRequests: int(cfg.Gateway.MaxConcurrentRequests.WithDefault(int64(c
RetrievalTimeout: cfg.Gateway.RetrievalTimeout.WithDefault(config.DefaultRetriev
}
```

3. 도메인 기반 접근 제어

```
{
"Gateway": {
"PublicGateways": {
"example.com": {
"Paths": ["/ipfs", "/ipns"],
"UseSubdomains": false,
"NoDNSLink": true
},
"trusted.example.com": {
"Paths": ["/ipfs", "/ipns", "/api"],
"UseSubdomains": true,
"DeserializedResponses": true
}
}
}
}
```

콘텐츠 필터링 및 차단 메커니즘

CID 기반 콘텐츠 차단

1. 블랙리스트 구현

```
// 차단된 CID 목록 관리
type ContentBlocker struct {
blockedCIDs  map[cid.Cid]BlockReason
mutex sync.RWMutex
}

type BlockReason struct {
Reason  string
Timestamp time.Time
Reporter  string
}

func (cb *ContentBlocker) IsBlocked(c cid.Cid) (bool, BlockReason) {
cb.mutex.RLock()
defer  cb.mutex.RUnlock()
if  reason, exists := cb.blockedCIDs[c];  exists {
return true, reason
}
```

```
return false, BlockReason{}

}


func (cb *ContentBlocker) BlockCID(c cid.Cid, reason  string, reporter  string)

cb.mutex.Lock()

defer  cb.mutex.Unlock()

cb.blockedCIDs[c] = BlockReason{

Reason: reason,

Timestamp: time.Now(),

Reporter: reporter,

}

}
```

2. 동적 콘텐츠 스캔

```
// 콘텐츠 스캔 인터페이스

type ContentScanner interface {

ScanContent(data []byte, contentType  string) (ScanResult, error)

}


type ScanResult struct {

IsBlocked  bool

Reason  string

Confidence  float64
```

```
Categories []string
}

// 실시간 콘텐츠 검사
func (gw *Gateway) scanAndFilter(data []byte, contentType string) error {
for _, scanner := range gw.contentScanners {
result, err := scanner.ScanContent(data, contentType)
if err != nil {
log.Warnf("Content scan error: %v", err)
continue
}
if result.IsBlocked && result.Confidence > 0.8 {
return fmt.Errorf("content blocked: %s", result.Reason)
}
}
return nil
}
```

피어 기반 접근 제어

1. 피어 블랙리스트 관리

```
type PeerBlocker struct {
blockedPeers map[peer.ID]BlockInfo
```

```
blockedIPs  map[string]BlockInfo
mutex sync.RWMutex
}


type BlockInfo struct {
Reason  string
BlockedAt time.Time
ExpiresAt *time.Time
Permanent  bool
}


func (pb *PeerBlocker) IsPeerBlocked(peerID peer.ID) bool {
pb.mutex.RLock()
defer  pb.mutex.RUnlock()
if  info, exists := pb.blockedPeers[peerID];  exists {
if  info.Permanent {
return true
}
if  info.ExpiresAt != nil && time.Now().After(*info.ExpiresAt) {
// 임시 차단 만료
delete(pb.blockedPeers, peerID)
return false
}
```

```
return true
}
return false
}
```

2. IP 주소 기반 차단

```
func (pb *PeerBlocker) IsIPBlocked(ip string) bool {
pb.mutex.RLock()
defer pb.mutex.RUnlock()
// 정확한 IP 매치
if info, exists := pb.blockedIPs[ip]; exists {
return pb.isBlockActive(info)
}
// CIDR 범위 확인
for cidr, info := range pb.blockedCIDRs {
if pb.ipInCIDR(ip, cidr) {
return pb.isBlockActive(info)
}
}
return false
}
```

지역 기반 접근 제어

1. 지리적 IP 차단

```
type GeoBlocker struct {
blockedCountries  map[string]bool
blockedRegions  map[string]bool
geoIPDB GeoIPDatabase
}

func (gb *GeoBlocker) IsLocationBlocked(ip  string) (bool, error) {
location, err := gb.geoIPDB.Lookup(ip)
if  err != nil {
return false, err
}
if  gb.blockedCountries[location.Country] {
return true, nil
}
if  gb.blockedRegions[location.Region] {
return true, nil
}
return false, nil
}
```

2. 시간대 기반 접근 제어

```
type TimeBasedAccessControl struct {
allowedHours  map[int]bool
timezone *time.Location
}

func (tbac *TimeBasedAccessControl) IsAccessAllowed() bool {
now := time.Now().In(tbac.timezone)
return  tbac.allowedHours[now.Hour()]
}
```

법적 준수 및 규정 대응

DMCA 및 저작권 보호

1. DMCA 신고 처리 시스템

```
type DMCANotice struct {
ID  string
ReportedCIDs []cid.Cid
Reporter ContactInfo
CopyrightOwner ContactInfo
Description  string
SubmittedAt time.Time
```

```
Status DMCAStatus
CounterNotice *CounterNotice
}

type DMCAStatus int

const (
DMCAPending DMCAStatus = iota
DMCAProcessed
DMCACountered
DMCARestored
)

func (dmca *DMCAHandler) ProcessNotice(notice *DMCANotice) error {
// 1. 신고 유효성 검증
if err := dmca.validateNotice(notice); err != nil {
return err
}
// 2. 콘텐츠 자동 차단
for _, c := range notice.ReportedCIDs {
dmca.contentBlocker.BlockCID(c, "DMCA takedown", notice.Reporter.Email)
}
// 3. 신고 기록 저장
```

```
dmca.notices[notice.ID] = notice

// 4. 알림 발송

dmca.notifyStakeholders(notice)

return nil

}
```

2. 이의 제기 처리

```
type CounterNotice struct {

OriginalNoticeID string

CounterClaimant ContactInfo

Justification string

SubmittedAt time.Time

SwornStatement bool

}


func (dmca *DMCAHandler) ProcessCounterNotice(counter *CounterNotice) error {

originalNotice, exists := dmca.notices[counter.OriginalNoticeID]

if !exists {

return errors.New("original notice not found")

}

// 이의 제기 기록

originalNotice.CounterNotice = counter
```

```
originalNotice.Status = DMCACountered

// 14일 후 자동 복원 스케줄링

dmca.scheduleRestoration(originalNotice, 14*24*time.Hour)

return  nil

}
```

GDPR 및 개인정보 보호

1. 개인정보 식별 및 제거

```
type PrivacyScanner struct {

patterns []PrivacyPattern

}


type PrivacyPattern struct {

Name  string

Regex *regexp.Regexp

Severity PrivacySeverity

Description  string

}


func (ps *PrivacyScanner) ScanForPII(data []byte) ([]PIIMatch, error) {

var  matches []PIIMatch

for  _, pattern := range  ps.patterns {
```

```
if pattern.Regex.Match(data) {
matches = append(matches, PIIMatch{
Pattern: pattern.Name,
Severity: pattern.Severity,
Description: pattern.Description,
})
}
}
return matches, nil
}
```

2. 데이터 삭제 요청 처리

```
type DataDeletionRequest struct {
RequestID string
DataSubject ContactInfo
CIDsToDelete []cid.Cid
Reason string
SubmittedAt time.Time
Status DeletionStatus
}

func (gdpr *GDPRHandler) ProcessDeletionRequest(req *DataDeletionRequest) error
// 1. 요청자 신원 확인
```

```
if err := gdpr.verifyDataSubject(req.DataSubject); err != nil {
return err
}
// 2. 개인정보 포함 여부 확인
for _, c := range req.CIDsToDelete {
if gdpr.containsPII(c) {
gdpr.contentBlocker.BlockCID(c, "GDPR deletion request", req.DataSubject.Email)
}
}
// 3. 삭제 요청 기록
gdpr.deletionRequests[req.RequestID] = req
return nil
}
```

우회 방지 및 보안 강화

해시 변형 감지

1. 유사 콘텐츠 탐지

```
type SimilarityDetector struct {
hashCache map[string][]cid.Cid
threshold float64
}
```

```
func (sd *SimilarityDetector) DetectSimilarContent(newCID cid.Cid, content []byt
// 퍼지 해시 계산
fuzzyHash := sd.calculateFuzzyHash(content)
var similarCIDs []cid.Cid
for existingHash, cids := range sd.hashCache {
similarity := sd.calculateSimilarity(fuzzyHash, existingHash)
if similarity > sd.threshold {
similarCIDs = append(similarCIDs, cids...)
}
}
return similarCIDs, nil
}
```

2. 메타데이터 기반 분석

```
type MetadataAnalyzer struct {
knownBadMetadata map[string]bool
}

func (ma *MetadataAnalyzer) AnalyzeMetadata(metadata map[string]interface{}) (b
// 파일 크기, 생성 시간, EXIF 데이터 등 분석
if fileSize, ok := metadata["size"].(int64); ok {
```

```
if ma.isSuspiciousSize(fileSize) {
return true, "suspicious file size pattern"
}
}
if exifData, ok := metadata["exif"].(map[string]interface{}); ok {
if ma.containsBadExifData(exifData) {
return true, "malicious EXIF data detected"
}
}
return false, ""
}
```

네트워크 행동 분석

1. 트래픽 패턴 모니터링

```
type TrafficAnalyzer struct {
requestPatterns map[peer.ID]*RequestPattern
anomalyThreshold float64
}

type RequestPattern struct {
RequestCount int
```

```
LastRequestTime time.Time

RequestedCIDs []cid.Cid

AnomalyScore  float64

}


func (ta *TrafficAnalyzer) AnalyzeRequest(peerID peer.ID, requestedCID cid.Cid)

pattern := ta.requestPatterns[peerID]

if  pattern == nil {

pattern = &RequestPattern{}

ta.requestPatterns[peerID] = pattern

}

// 요청 패턴 업데이트

pattern.RequestCount++

pattern.LastRequestTime = time.Now()

pattern.RequestedCIDs = append(pattern.RequestedCIDs, requestedCID)

// 이상 점수 계산

pattern.AnomalyScore = ta.calculateAnomalyScore(pattern)

return  pattern.AnomalyScore > ta.anomalyThreshold, nil

}
```

2. 분산 차단 정보 공유

```
type BlocklistSyncer struct {
```

```
peers []peer.ID

localBlocks  map[cid.Cid]BlockInfo

remoteBlocks  map[cid.Cid]map[peer.ID]BlockInfo

}



func (bs *BlocklistSyncer) SyncBlocklists() error {

for  _, peerID := range  bs.peers {

remoteBlocks, err := bs.fetchRemoteBlocklist(peerID)

if  err != nil {

log.Warnf("Failed to sync blocklist from %s: %v", peerID, err)

continue

}

// 신뢰할 수 있는 피어의 차단 정보 적용

for  blockedCID, blockInfo := range  remoteBlocks {

if  bs.shouldTrustPeer(peerID) && bs.isConsensusReached(blockedCID) {

bs.localBlocks[blockedCID] = blockInfo

}

}

}

return  nil

}
```

모니터링 및 감사

보안 이벤트 로깅

1. 접근 로그 기록

```
type AccessLog struct {
Timestamp time.Time
ClientIP  string
PeerID peer.ID
RequestPath  string
Method  string
StatusCode  int
UserAgent  string
Blocked  bool
BlockReason  string
}

func (al *AccessLogger) LogRequest(req *http.Request, resp *http.Response, block
log := AccessLog{
Timestamp: time.Now(),
ClientIP: getClientIP(req),
RequestPath: req.URL.Path,
Method: req.Method,
StatusCode: resp.StatusCode,
UserAgent: req.UserAgent(),
```

```
Blocked: blocked,

BlockReason: reason,

}

al.writeLog(log)

}
```

2. 보안 메트릭 수집

```
type SecurityMetrics struct {

BlockedRequests prometheus.Counter

BlockedPeers prometheus.Gauge

DMCANotices prometheus.Counter

GDPRRequests prometheus.Counter

AnomalousTraffic prometheus.Counter

}


func (sm *SecurityMetrics) RecordBlockedRequest(reason  string) {

sm.BlockedRequests.With(prometheus.Labels{"reason": reason}).Inc()

}
```

정기 보안 감사

1. 자동화된 보안 검사

```go
func (sa *SecurityAuditor) RunDailyAudit() error {
// 1. 차단 목록 유효성 검사
if err := sa.validateBlocklists(); err != nil {
return err
}
// 2. 접근 패턴 분석
if err := sa.analyzeAccessPatterns(); err != nil {
return err
}
// 3. 설정 무결성 검사
if err := sa.validateSecurityConfig(); err != nil {
return err
}
// 4. 보고서 생성
report := sa.generateSecurityReport()
sa.sendReport(report)
return nil
}
```

이러한 종합적인 접근 제어 및 콘텐츠 차단 시스템을 통해 IPFS는 분산 환경에서도 효과적인 보안 정책을 구현하고 법적 요구사항을 준수할 수 있습니다.

참고 자료

- 암호화 시스템 및 키 관리
- 신원 관리 및 데이터 무결성
- HTTP API 시스템
- 게이트웨이 구현
- 설정 관리 시스템